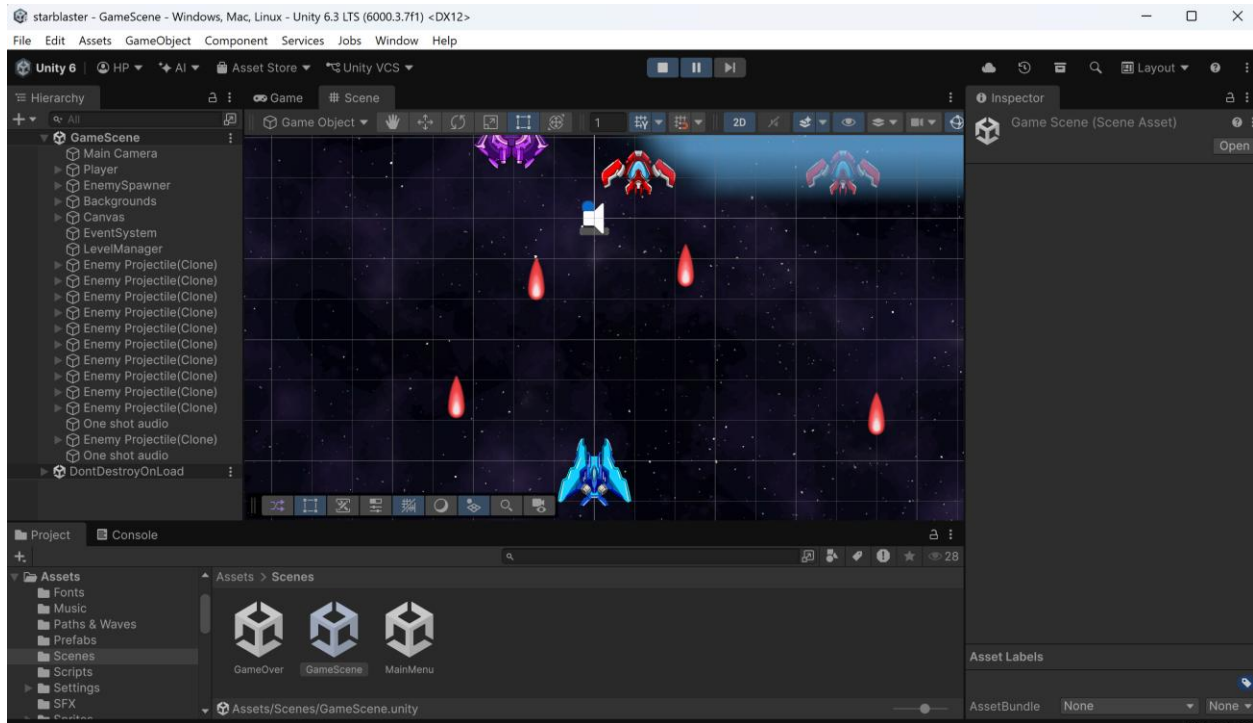


Lab 03 - Tìm hiểu về dự án Star Blaster



PHẦN 2: Khám phá cấu trúc dự án

Task 2.1: Cấu trúc thư mục

Câu 1: Liệt kê tất cả các thư mục con trực tiếp trong Assets/. Mô tả ngắn gọn vai trò của từng thư mục

1. Fonts – Chứa các font chữ được sử dụng trong trò chơi
2. Music – Lưu trữ các file nhạc nền
3. Paths & Waves – Chứa dữ liệu về đường đi và sóng cho các đối tượng trong trò chơi
4. Prefabs – Lưu trữ các mẫu sẵn của các game objects có thể tái sử dụng
5. SFX – Chứa các file âm thanh hiệu ứng
6. Scenes – Lưu trữ các cảnh chơi của trò chơi
7. Scripts – Chứa tất cả file code C# của dự án
8. Settings – Lưu trữ các cài đặt cấu hình của dự án
9. Sprites – Chứa các hình ảnh 2D được sử dụng trong trò chơi

10. TextMesh Pro – Thư mục hỗ trợ TextMesh Pro (thư viện UI/Text nâng cao cho Unity)

Câu 2: Dự án có bao nhiêu Scene? Liệt kê tên và mô tả vai trò của từng Scene.

Dự án có 3 Scenes:

MainMenu.unity: Màn hình menu chính – nơi người chơi bắt đầu trò chơi , có các tùy chọn như : chơi , thoát game

GameScenes.unity: Màn hình chơi chính – Nơi diễn ra gameplay thực tế với nhân vật chính, địch, các cơ chế của trò chơi

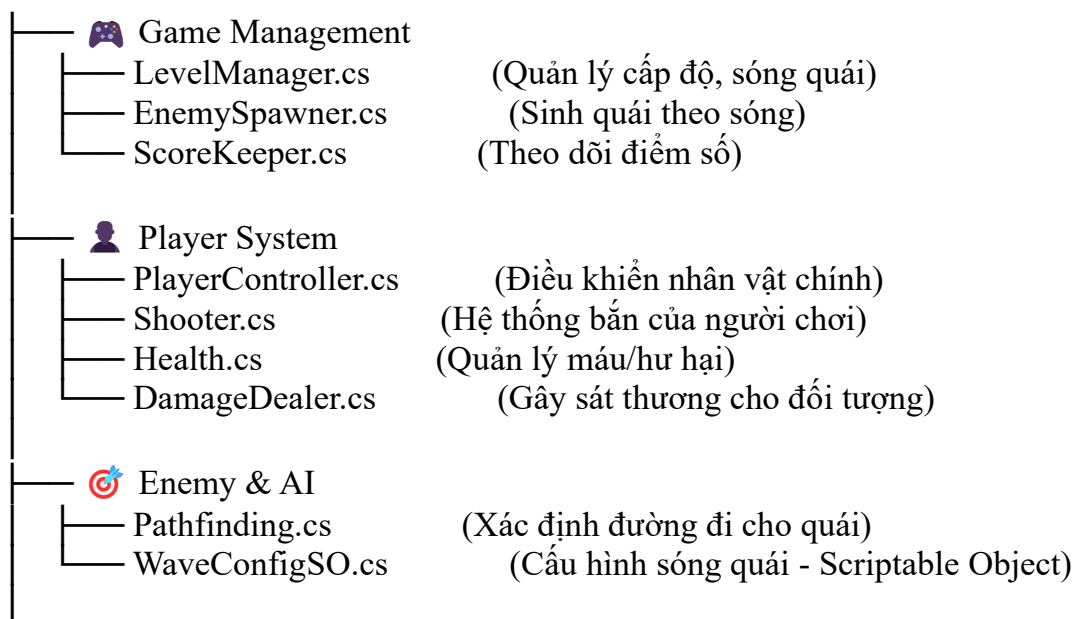
GameOver.unity: Màn hình kết thúc trò chơi – Hiển thị kết quả trận đấu (thắng / thua) , điểm số và các tùy chọn chơi lại hoặc quay về menu chính

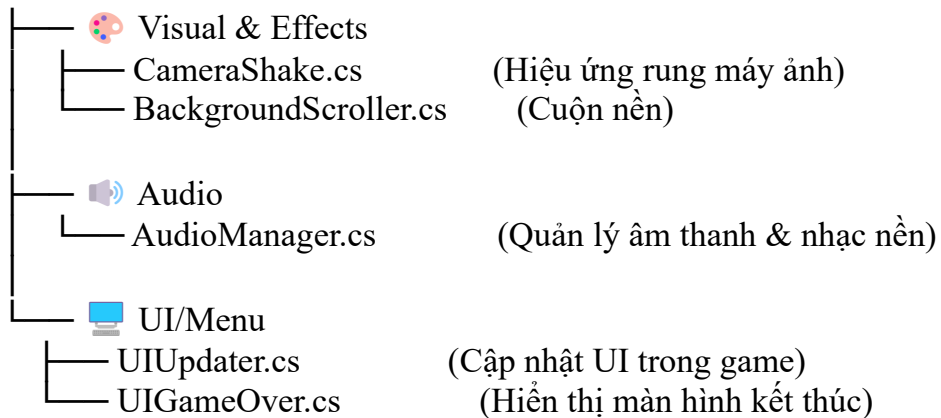
Luồng chơi: MainMenu.unity → GameScenes.unity → GameOver.unity → quay lại MainMenu.unity hoặc chơi lại

Câu 3: Thư mục Assets/Scripts/ được tổ chức như thế nào? Vẽ sơ đồ cây thư mục (tree) của thư mục Scripts/ .

Thư mục **Assets/Scripts/** được tổ chức theo **cấu trúc phẳng** (Flat Structure) - tất cả các file C# được đặt trực tiếp trong một thư mục chung, không có thư mục con phân loại.

Assets/Scripts/





Phân tích tổ chức:

Nhóm	Số lượng	Mục đích
Game Management	3	Quản lý luồng game, tạo quái, tính điểm
Player System	4	Điều khiển, bắn, máu, hư hại
Enemy & AI	2	Xác định đường đi quái, cấu hình sóng
Visual & Effects	2	Hiệu ứng hình ảnh
Audio	1	Quản lý âm thanh
UI/Menu	2	Giao diện người dùng
Total	14 C# Scripts	

Câu 4: Mở thư mục Assets/Prefabs/ . Liệt kê các thư mục con và cho biết mỗi thư mục chứa loại prefab nào.

Player : Nhân vật chính

Player Projectile : Đạn người chơi

Enemy Projectile : Đạn quái

Enemy : Quái cơ bản (yếu)

TankyEnemy: Quái bèn (nhiều máu)

Explosion Particles : Hiệu ứng nổ tung

Backgrounds : Hình nền di chuyển

AudioManager : Quản lý âm thanh

LevelManager : Quản lý level/song

ScoreKeeper : Quản lý tổng điểm

Task 2.2: Scene và Hierarchy (15 phút)

Câu 6: Liệt kê tất cả các GameObject gốc (root) trong tab Hierarchy.

Tên GameObject	Loại	Mục đích
Main Camera	Camera	Máy ảnh chính - Render scene
Player	Prefab Instance	Nhân vật người chơi (điều khiển, bắn, máu)
EnemySpawner	Script GameObject	Spawn quái theo sóng (4 wave configs)
Backgrounds	Prefab Instance	Nền game cuộn
AudioManager	Prefab Instance	Quản lý âm thanh
ScoreKeeper	Prefab Instance	Quản lý tổng điểm
Canvas	UI Element	Giao diện người dùng (UI)
EventSystem	Event Handler	Xử lý sự kiện Input/UI
LevelManager	Prefab Instance	Quản lý cấp độ/sóng

Câu 7: Click chọn nhân vật **Player** trong Hierarchy → xem tab Inspector. Liệt kê tất cả các Component được gắn trên nhân vật.

1. Transform : Quản lí vị trí, xoay, scale của object
2. Sprite Renderer : Hiển thị sprite của nhân vật
3. Rigidbody 2D : Vật lý 2D – chuyển động , va chạm
4. Capsule Collider 2D : Xác định vùng va chạm (hình dạng capsule)
5. Health : Quản lí máu/HP của nhân vật
6. PlayerController : Điều khiển chuyển động của nhân vật
7. Shooter : Hệ thống bắn của nhân vật

TÌM HIỂU VỀ HỆ THỐNG GAME CỦA DỰ ÁN

1. UI Canvas System (UIUpdater.cs)

```
using TMPro;
using UnityEngine;
using UnityEngine.UI;

0 references | ☑ Unity Script (1 asset reference)
public class UIUpdater : MonoBehaviour
{
    [Header("Health")]
    2 references | ☑ Unity Serialized Field
    [SerializeField] Slider healthSlider;
    2 references | ☑ Unity Serialized Field
    [SerializeField] Health playerHealth;

    [Header("Score")]
    1 reference | ☑ Unity Serialized Field
    [SerializeField] TextMeshProUGUI scoreText;
    2 references
    ScoreKeeper scoreKeeper;

    0 references | ☑ Unity Message
    void Start()
    {
        scoreKeeper = FindFirstObjectByType<ScoreKeeper>();
        healthSlider.maxValue = playerHealth.GetHealth();
    }

    0 references | ☑ Unity Message
    void Update()
    {
        scoreText.text = scoreKeeper.GetScore().ToString("000000000");
        healthSlider.value = playerHealth.GetHealth();
    }
}
```

Chức năng:

Cập nhật **Health Slider** mỗi frame từ Health script

Cập nhật **Score Text** với định dạng 9 chữ số (000000000)

Khởi tạo healthSlider.maxValue từ máu ban đầu của Player

Thành phần	Mục đích
Slider	Thanh máu (HP Bar)
TextMeshProUGUI	Hiển thị điểm số
Health script	Lấy máu hiện tại
ScoreKeeper script	Lấy điểm số hiện tại

2. Audio Manager System (AudioManager.cs)

Đặc điểm:

Singleton Pattern - Chỉ tồn tại 1 instance trên toàn game

DontDestroyOnLoad - Giữ âm thanh qua scene

AudioSource.PlayClipAtPoint - Phát âm ở vị trí camera

Âm thanh	Sử dụng khi
shootingClip	Bắn đạn (Player hoặc Enemy)
damageClip	Bị hư hại (Player hoặc Enemy)

3. Enemy Spawn System (EnemySpawner.cs)

```

public class EnemySpawner : MonoBehaviour
{
    1 reference | ☒ Unity Serialized Field
    [SerializeField] WaveConfigSO[] waveConfigs;
    1 reference | ☒ Unity Serialized Field
    [SerializeField] float timeBetweenWaves = 1f;
    1 reference | ☒ Unity Serialized Field
    [SerializeField] bool isLooping;
    6 references
    WaveConfigSO currentWave;

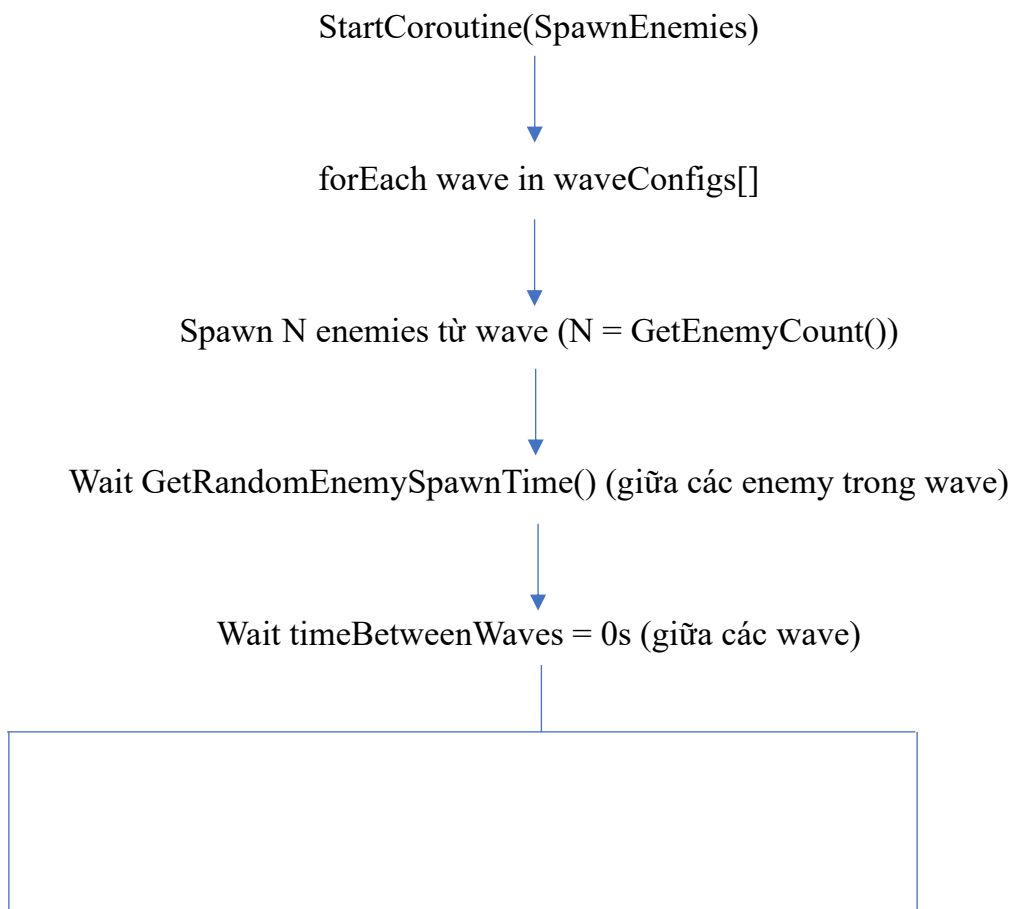
    0 references | ☒ Unity Message
    void Start()
    {
        StartCoroutine(SpawnEnemies());
    }

    1 reference
    IEnumerator SpawnEnemies()
    {
        do
        {
            foreach (WaveConfigSO wave in waveConfigs)
            {
                currentWave = wave;
                for (int i = 0; i < currentWave.GetEnemyCount(); i++)
                {
                    Instantiate(
                        currentWave.GetEnemyPrefab(i),
                        currentWave.GetStartingWaypoint().position,
                        Quaternion.identity,
                        transform);
                    yield return new WaitForSeconds(currentWave.GetRandomEnemySpawnTime());
                }
                yield return new WaitForSeconds(timeBetweenWaves);
            } while (isLooping);
        }
    }

    1 reference
    public WaveConfigSO GetCurrentWave()
    {
        return currentWave;
    }
}

```

Luồng spawn :



isLooping=true

Lặp lại

isLooping=false

Kết thúc

Scriptable Object (WaveConfigSO):

```
[CreateAssetMenu(fileName = "WaveConfig", menuName = "New WaveConfig")]
public class WaveConfigSO : ScriptableObject
{
    [SerializeField] GameObject[] enemyPrefabs;    // Các loại quái
    [SerializeField] Transform pathPrefab;         // Đường đi
    [SerializeField] float enemyMoveSpeed = 5f;    // Tốc độ
    [SerializeField] float timeBetweenEnemySpawns = 1f; // Khoảng cách spawn
    [SerializeField] float enemySpawnVariance = 0f; // Biến động
    [SerializeField] float minimumSpawnTime = 0.2f; // Tối thiểu

    public float GetRandomEnemySpawnTime()
    {
        // Random với khoảng từ (1s - 0s) đến (1s + 0s) = 1s
        float spawnTime = Random.Range(timeBetweenEnemySpawns - enemySpawnVariance,
            timeBetweenEnemySpawns + enemySpawnVariance);

        // Đảm bảo không quá nhanh
        spawnTime = Mathf.Clamp(spawnTime, minimumSpawnTime, float.MaxValue);

        return spawnTime;
    }
}
```

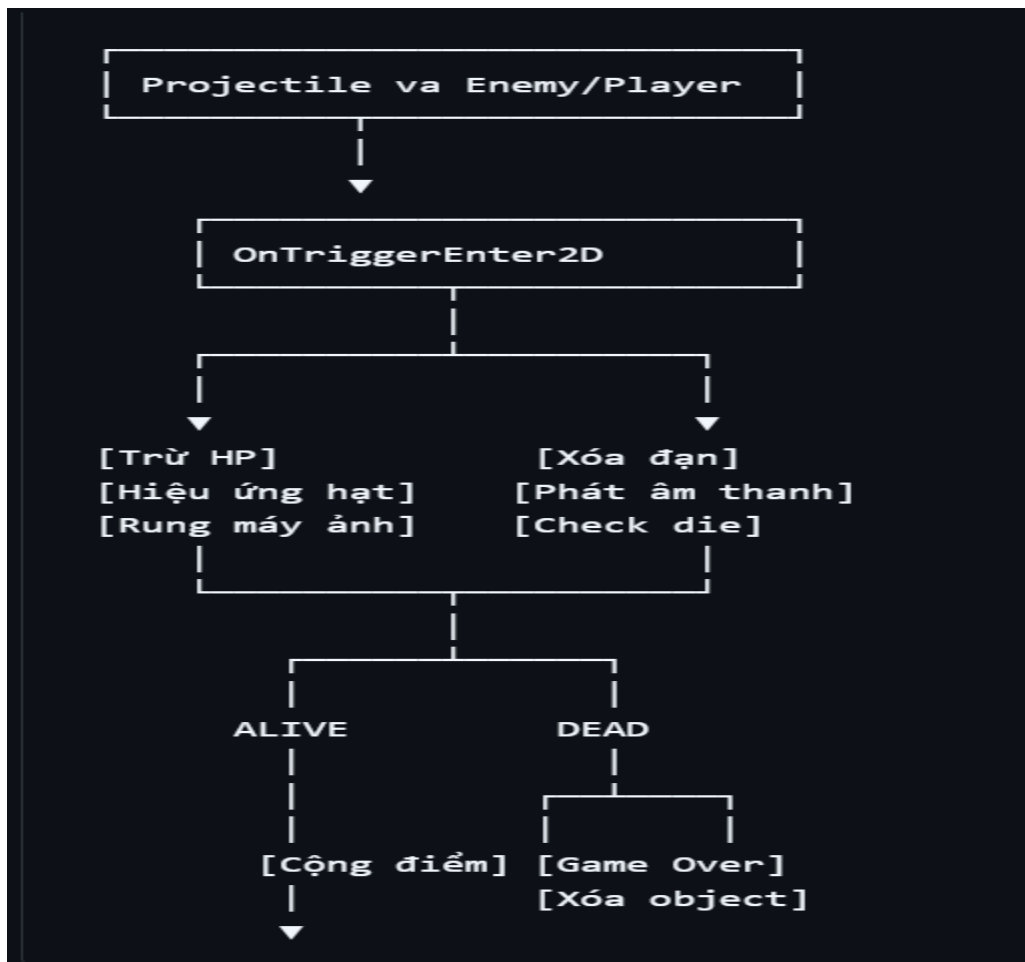
Config trong GameScene:

4 Waves lần lượt spawn

timeBetweenWaves = 0s (liên tục giữa các wave)

isLooping = 1 (lặp lại các wave)

4. Damage & Collision System (DamageDealer.cs + Health.cs)



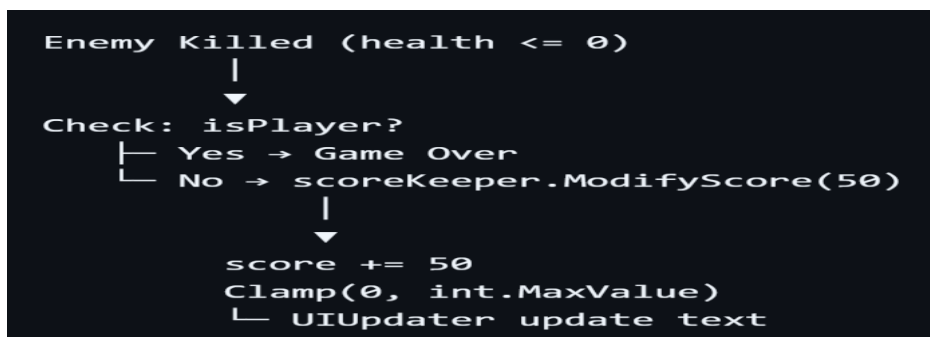
5. Scoring System (ScoreKeeper.css)

Cộng điểm:

Enemy bình thường: 50 điểm

Enemy đặc biệt (TankyEnemy) :150 điểm

Luồng tính điểm:



→ Quy trình gameplay:

1. **KHỞI ĐỘNG** → EnemySpawner bắt đầu spawn sóng quái
2. **CHƠI GAME** → Player bắn đạn vào Enemy
3. **VA CHẠM** → DamageDealer va Enemy.Health
4. **HẬU QUẢ** →
 - Enemy chết → Cộng điểm → UIUpdater cập nhật Score Text
 - Player chết → Game Over
5. **HIỆU ỨNG** → Particle, âm thanh, rung máy ảnh đồng thời